
CS614 - Endsem Project Review

Aditi Khandelia Arush Upadhyaya Kushagra Srivastava Sankalp Mittal Vidhi Jain

April 22, 2026

Project Overview

Iterative pre-copy live migration transfers memory pages from a running process to a destination host while the process continues to execute, repeating the transfer for pages that were dirtied during the previous round until the remaining working set is small enough to complete migration with a brief final pause. The efficiency of this scheme depends entirely on the ability to identify, at the end of each round, *exactly* which pages were written. For CPU memory, the Linux kernel provides this capability through soft-dirty bits embedded in page-table entries, readable via `/proc/PID/pagemap`. No analogous mechanism exists for GPU-managed memory. NVIDIA's Unified Virtual Memory (UVM) subsystem, which underpins `cudaMallocManaged` and handles demand-paging between host and device, exposes no interface for querying which pages a process has modified. Consequently, any migration or checkpointing system must conservatively copy the *entire* GPU memory footprint on every iteration, rendering iterative pre-copy migration of large GPU workloads impractical.

This project addresses that gap by implementing **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module. UVM already intercepts every GPU page fault to manage demand-paging; we augment the write-fault servicing path to record each written page in an in-kernel data structure, without introducing any additional hardware overhead. A **procfs**-based interface allows a privileged user to start and stop tracking epochs, perform atomic cutover snapshots, pause and resume recording, and retrieve the set of pages written by a given process with **page-level granularity** and a per-page **nanosecond-resolution timestamp** suitable for ordering events across migration rounds.

The core mechanism exploits the GPU's demand-paging fault path. At the start of each tracking epoch, write permissions are *downgraded* from all currently mapped GPU pages, leaving them readable but not writable. Any subsequent GPU write generates a replayable fault; the driver's fault-servicing path records the faulting page and timestamp before restoring write permission and replaying the instruction. Pages that are only read never fault and are not recorded, eliminating the overhead of tracking read-only accesses. Formal determinism guarantees ensure that no write escapes recording and that the boundary between consecutive migration rounds is precisely defined through a barrier-protected atomic table swap.

Group Details

Following are the details of the group members:

Name	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in
Sankalp Mittal	220963	sankalpm22@iitk.ac.in
Vidhi Jain	221191	vidhijain22@iitk.ac.in

Implementation

End-to-end Workflow

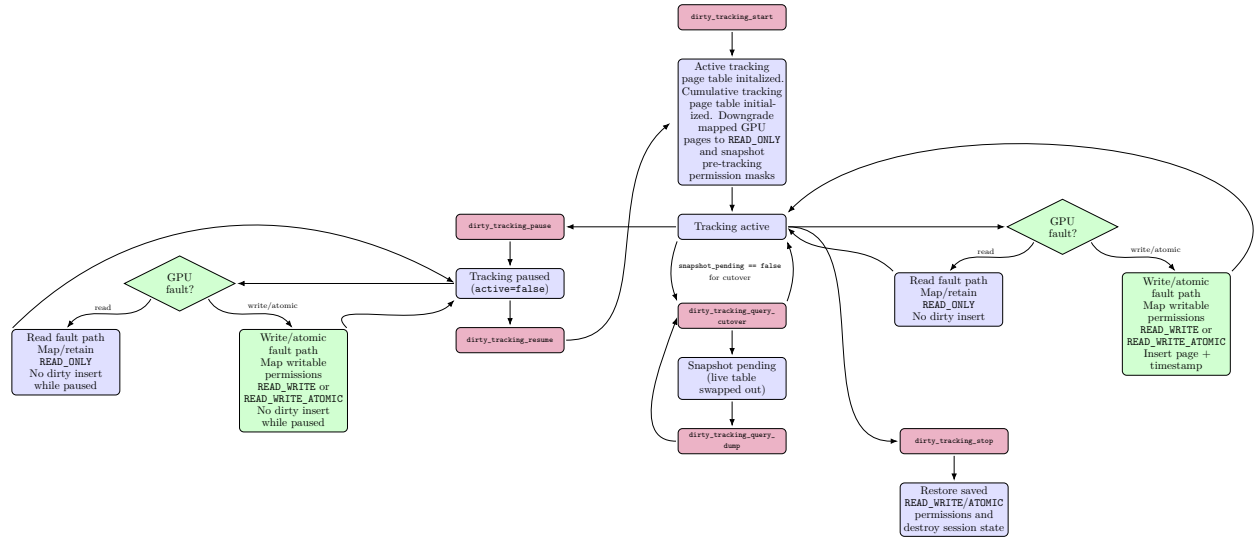


Figure 1: End-to-end workflow of the dirty page tracking system, illustrating procs control paths, GPU fault handling, and state transitions.

State Machine

Dirty tracking is modelled as a single-owner lifecycle state machine, with all lifecycle operations serialized by `uvm_dirty_lifecycle_lock` and guarded by owner-PID checks. The states are:

1. **Tracking stopped.** `started=false`, `active=false`, `snapshot_pending=false`.
2. **Tracking active with snapshot.** `started=true`, `active=true`, `snapshot_pending=true`.
3. **Tracking active without snapshot.** `started=true`, `active=true`, `snapshot_pending=false`.
4. **Tracking paused with snapshot.** `started=true`, `active=false`, `snapshot_pending=true`.
5. **Tracking paused without snapshot.** `started=true`, `active=false`, `snapshot_pending=false`.

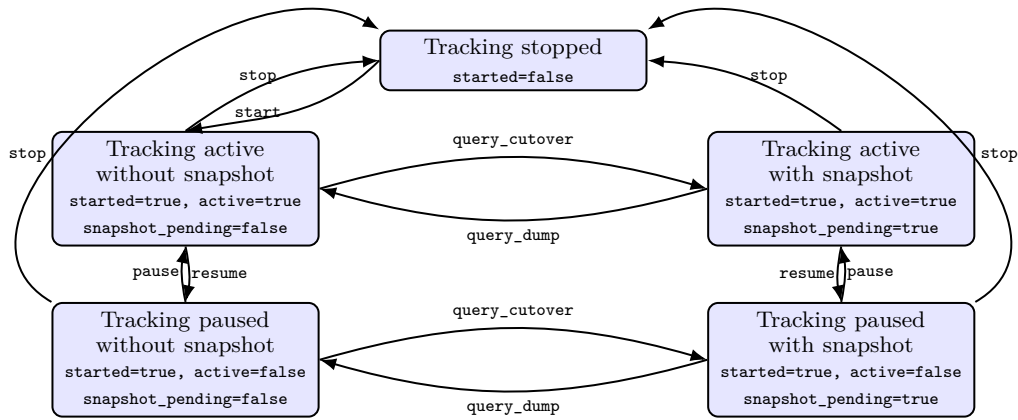


Figure 2: State-transition diagram for dirty-tracking lifecycle states.

Transitions that violate owner checks, state preconditions, or snapshot guards are rejected by the procfs handlers.

Permission Lifecycle

Figure 1 captures the permission policy used to turn hardware write access into explicit software-recorded dirty events.

Start/Resume phase At `start` (and again at `resume`), the implementation walks the owner VA space and revokes GPU write-class permissions from mapped pages, forcing tracked mappings to `READ_ONLY`. Before revocation, per-page write/atomic masks are snapshot so `stop` can restore exact pre-tracking permissions.

Active tracking phase During replayable fault servicing:

1. Read faults remain mapped as `READ_ONLY` and are not inserted into the dirty set.
2. Write/atomic faults are serviced with writable permissions (`READ_WRITE` or `READ_WRITE_ATOMIC`), and the page number plus timestamp are inserted into the live dirty table.

This enforces first-write capture while preserving normal execution after fault resolution.

Paused phase When paused, dirty recording is disabled, but fault handling still progresses: read paths remain `READ_ONLY`, while write/atomic paths may regain writable permissions for correctness and forward progress. Because `active=false`, no dirty inserts are emitted in this interval.

Cutover/Dump phase `query_cutover` requires `snapshot_pending==false`. Under the query barrier, it swaps out the live table into a frozen snapshot and installs a new live table so recording can continue. `query_dump` then consumes that frozen snapshot: in `delta` mode it emits snapshot contents directly; in `cumulative` mode it merges snapshot entries into the cumulative table before output.

Stop phase `stop` restores the previously captured per-page write/atomic permission masks and tears down live, snapshot, and cumulative tracking state, returning the system to its pre-tracking permission configuration.

Modified Driver Files

All driver modifications reside under `kernel-open/nvidia-uvm/`. The files containing functionality added or significantly modified by us are:

1. **uvm_common.h / uvm_common.c**
The primary implementation file. Defines `struct dirty_page_info` (page number, timestamp) and the three global dirty-set tables (`g_dirty_ds`, `g_dirty_ds_snapshot`, `g_dirty_ds_cumulative`). Implements table lifecycle (`uvm_dirty_page_table_init`, `uvm_dirty_page_table_destroy`), fault recording (`uvm_dirty_page_table_record`), snapshot management (`uvm_dirty_new_snapshot`, `uvm_dirty_merge_snapshot_into_cumulative_locked`), and all eight procfs handler implementations including `uvm_dirty_procfs_init`. Exports function pointers (`uvm_dirty_invalidate_fn`, `uvm_dirty_activate_now_fn`, `uvm_dirty_barrier_end_fn`, `uvm_dirty_query_barrier_begin_fn`, `uvm_dirty_query_barrier_end_fn`) that are registered from `uvm_va_space.c` at module init.
2. **uvm_dirty_ds.h**
Defines the `uvm_dirty_ds_ops` interface (`init`, `insert`, `lookup`, `for_each_in_range`, `destroy`), the `uvm_dirty_ds` struct holding ops pointer, backend-private data, and statistics, and inline timing wrappers for all operations.
3. **uvm_dirty_ds_xarray.c**
xarray backend using atomic `xa_cmpxchg` for first-write-wins insert.
4. **uvm_dirty_ds_vector.c**
Sorted dynamic array backend with binary-search insert and lookup.
5. **uvm_dirty_ds_vector_unsorted.c**
Unsorted append-order vector with spinlock, preserving fault-arrival order.

6. **uvm_dirty_ds_linked_list.c**
Singly-linked list with sentinel node.
7. **uvm_dirty_ds_bitmap.c**
Flat bit array covering 2^{38} pages (32 MB fixed allocation); O(1) operations; timestamps not stored.
8. **uvm_dirty_ds_nested_bitmap.c**
Two-level hierarchical bitmap for sparse page sets.
9. **uvm_dirty_ds_chunked.c**
Chunked vector with a background kthread sorter for deferred ordering.
10. **uvm_gpu_replayable_faults.c**
The write-fault hook. After standard fault servicing, checks `uvm_dirty_tracking_active()` and `service_access_type >= UVM_FAULT_ACCESS_TYPE_WRITE`, then calls `uvm_dirty_page_table_record` with the faulting page number and `ktime_get_ns()` timestamp.
11. **uvm_va_space.c**
Implements the barrier infrastructure. `uvm_dirty_downgrade_all_permissions` walks all va-ranges and va-blocks for the owner's va_space, calling `uvm_va_block_revoke_prot_mask` with `UVM_PROT_READ_WRITE` to strip write access from all GPU PTEs, followed by `uvm_tracker_wait` to confirm GPU processing. `uvm_dirty_query_barrier_begin` and `uvm_dirty_query_barrier_end` implement the cutover barrier by locking all parent GPU replayable-fault ISRs and spinning until the fault buffer is empty. `uvm_va_space_dirty_init` registers all five function pointers at module init.
12. **uvm_va_space.h**
Declares `uvm_va_space_dirty_init`, the registration hook invoked during module initialisation.
13. **uvm_va_block.c**
`compute_new_permission` modified to force `READ_ONLY` on read faults when tracking is active for the faulting va_space's owner, preventing read faults from obtaining write mappings that would bypass recording.
14. **uvm_migrate.c**
`block_migrate_map_unmapped_pages` modified to map GPU-migrated pages as `READ_ONLY` when tracking is active, closing the `cudaMemPrefetchAsync` bypass.
15. **uvm_va_block.h**
Extends per-GPU va-block state with `dirty_downgraded_pages` and `dirty_downgraded_atomic_pages` masks used to restore exact pre-tracking permissions at stop.
16. **uvm_procfs.c**
Integrates the dirty-tracking procfs interface into `/proc/driver/nvidia-uvm/` by invoking `uvm_dirty_procfs_init(uvm_proc_dir)` during procfs setup.
17. **nvidia-uvm-sources.Kbuild**
Adds all dirty data-structure backend source files (`uvm_dirty_ds_xarray.c`, `uvm_dirty_ds_bitmap.c`, `uvm_dirty_ds_vector.c`, `uvm_dirty_ds_vector_unsorted.c`, `uvm_dirty_ds_chunked.c`, `uvm_dirty_ds_nested_bitmap.c`, `uvm_dirty_ds_linked_list.c`) to the module build list.
18. **uvm.c**
Top-level integration; calls `uvm_va_space_dirty_init` during module initialisation so callback pointers are registered before tracking control paths execute.

Procfs Interface

Eight procfs entries are created under `/proc/driver/nvidia-uvm/`:

1. **dirty_tracking_start (write)**
Input: "`<pid> <delta|cumulative>`"
Initializes a fresh live table for the target PID, selects query mode (`delta` or `cumulative`), executes

permission downgrade over the owner's managed GPU mappings, and finally enables recording. The request is rejected if tracking is already started, if the input format is invalid, or if initialization/callback setup fails.

2. **dirty_tracking_stop (write)**

Input: "<pid>"

Restores previously downgraded GPU page permissions, destroys all tracking state (live/snapshot/cumulative), clears snapshot-pending state, and flushes accumulated DS statistics to `/tmp/uvdm_dirty_ds_stats.txt`.

The request is rejected if the supplied PID is not the active owner.

3. **dirty_tracking_pause (write)**

Input: "<pid>"

Establishes the query barrier, sets `active=false`, and then releases the barrier. This ensures pause is synchronized with replayable-fault ISR state.

The request is rejected if the PID does not match the active owner or if tracking is not currently active.

4. **dirty_tracking_resume (write)**

Input: "<pid>"

Re-executes the same downgrade/activate/barrier-end sequence used by `start`, re-establishing deterministic write-fault capture after a paused interval.

5. **dirty_tracking_query_cutover (write)**

Input: "<pid>"

Acquires the query barrier, atomically swaps `live` → `snapshot` under write lock, installs a fresh live table, updates epoch metadata, sets `g_snapshot_pending=true`, and releases the barrier.

The request is rejected if the previous snapshot has not yet been consumed by `query_dump`.

6. **dirty_tracking_query_dump (read)**

No input payload (read endpoint). Requires `g_snapshot_pending=true`.

In `delta` mode, emits only the current frozen snapshot. In `cumulative` mode, first merges snapshot contents into `g_dirty_ds_cumulative` (first-write-wins), then emits the cumulative union.

Output includes a header line followed by one line per page: `0xaddr timestamp_ns`.

Snapshot consumption is overflow-safe: when `seq_file` overflows, snapshot state is retained and replayed on the next read.

7. **dirty_ds_stats_toggle (write)**

Input: "enable" or "disable"

Enables or disables instrumentation for DS and callback timing counters. Writing `enable` also resets all counters/timers to zero to provide a clean measurement window.

8. **dirty_ds_stats (read)**

Read-only statistics snapshot for the current session.

Reports operation counts and average latency for init/insert/lookup/for-each/destroy, average lock-wait components, and callback timing for invalidate, activate-now, and barrier-end.

Design Choices

Determinism Guarantees

Data Structures

Global state Three `uvdm_dirty_ds` instances hold the tracking data:

1. `g_dirty_ds` - live table; fault handler inserts here.
2. `g_dirty_ds_snapshot` - frozen copy created by cutover; exists only between a cutover and the subsequent dump.
3. `g_dirty_ds_cumulative` - session-union table used in cumulative mode; merge-accumulates snapshot contents at each dump.

Backend abstraction All three tables share the `uvm_dirty_ds_ops` interface (`uvm_dirty_ds.h`), which provides `init`, `insert`, `lookup`, `for_each_in_range`, and `destroy`. Inline wrappers measure operation latency when statistics are enabled. The active backend is selected at compile time by setting the `.ops` pointer in the static `g_dirty_ds` initialiser.

Concurrency Design

A read-write semaphore (`uvm_dirty_ds_rwsem`) protects all three tables. Fault-handler insertions and lookups take the read side concurrently. Lifecycle operations (`init`, `destroy`, `cutover swap`) take the write side exclusively, blocking new insertions only during the brief pointer swap. A separate mutex (`uvm_dirty_lifecycle_lock`) serialises the procs lifecycle handlers (`start`, `stop`, `pause`, `resume`, `cutover`) against each other.

Testing and Analysis

Future Work

Source Code

Development Environment

Development and testing are conducted on a dedicated virtual machine (`dirtyuvm`) running:

- Linux kernel version 6.8 (custom build)
- Modified NVIDIA UVM driver version 580.65.06 (loaded as a kernel module)

Repository

The full source code is hosted at:

<https://github.com/aleph-5/open-gpu-kernel-modules>

This is a fork of NVIDIA's open-source GPU kernel modules, with our modifications applied on top.

Instructions for Run

Declaration

We declare that the work presented in this report is our own, except where otherwise acknowledged. We have also read and understood the University's policy on plagiarism and academic integrity, and we confirm that this report complies with those standards.